6G Slicing — Two-Tier Control Plane: Analysis Results

Prototype report. Dataset: Farreras et al. 2024 (Zenodo 10610616). Analytic capacity model + per-slice LLM/MCP compiler. mistral-large-4 excluded (cost, same decisions).

Caveat: single run, no repetitions. Non-LLM baselines are deterministic; each LLM result is one closed-loop trajectory per model per scenario, not an average over repeated runs. No error bars — variance was not measured.

1. Dataset (EDA)

100 samples. Aggregate load 47–810 Mbps (mean 376).

Head (first 10 samples):

eMBB Mbps	URLLC Mbps	mMTC Mbps	delay	loss
36	7	4	0.02189	0.567
36	32	5	0.03590	0.572
36	24	14	0.02249	0.634
54	21	2	0.01953	0.361
36	46	2	0.00242	0.005
72	21	2	0.00912	0.057
36	60	3	0.00641	0.047
36	49	14	0.00378	0.081
72	24	26	0.01010	0.338
72	49	6	0.00335	0.058

Per-slice load:

slice	min load	max load	mean load	% of total
eMBB	36	378	174	46%
URLLC	7	382	170	45%
mMTC	2	132	32	9%

KPI: delay min=0.00021 max=0.03590 mean=0.00488 | loss min=0.005 max=0.843 mean=0.253

2. Tier-1 components + KPI budgets

• **Capacity model** — analytic, deterministic (not ML); the scaling math every other part trusts:

```
effective_capacity = replicas * cap_per_replica
rho = load / capacity
delay = base_delay / (1 - rho)  if rho < 1 else max_delay
loss  = (load - capacity) / load if overloaded else 0
violation = delay > delay_budget OR loss > loss_budget
required_replicas = ceil(load / (cap * target_util))
```

• calibrated from the slice's P95 load (cap) and the dataset's min delay (base_delay); feeds the env, oracle, gate, baselines and MCP tools.

• **Forecaster** — learned. Predicts next aggregate load: linear regression over the last 5 points (or persistence). Output: forecast_mbps.

• **KPI predictor** — learned. Numpy MLP regressor (16 hidden, ReLU, MSE) trained on aggregate load → delay. p_violation = sigmoid((pred_delay − budget)/budget).

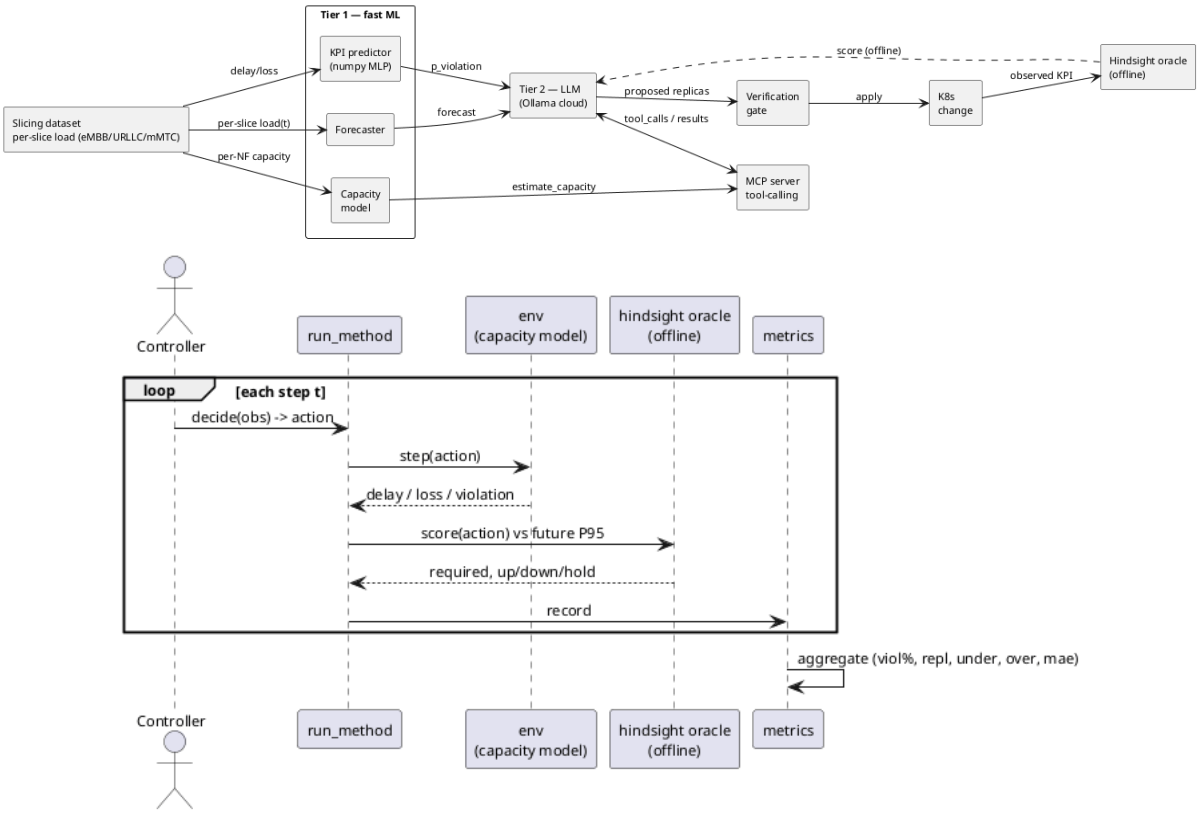
slice	cap/replica Mbps	delay_budget (x base)	loss_budget
eMBB	98	30x	0.02
URLLC	101	4x	0.005
mMTC	29	15x	0.005

3. Pipeline + harness

• dataset sample -> per-slice load -> Tier1 (forecaster + MLP) -> payload -> LLM (or capacity-rule) -> gate -> apply

• hindsight oracle (offline): future-load P95 -> required replicas, scores under/over-scaling

• every controller is decide(obs) -> action; run_method feeds it to env + oracle; metrics() aggregates



4. Methods

- **threshold** — reactive: scale only after a violation.
- **hpa** — ideal controller, always required_replicas(load).
- **predict-then-threshold** — MLP delay prediction then threshold (degenerates to HPA).
- **capacity-rule** — deterministic rule, near-optimal reference (shares the capacity model).
- **llm-real** — Ollama cloud LLM acting through MCP tools (tool-call-as-answer).

5. Metrics

Metric	Why it matters
SLA violation %	Outcome: how often delay/loss exceeds the slice budget (the cost the operator cares about).
mean replicas	Resource cost: over-provisioning wastes compute.
under-scaling	Missed scale-ups (replicas below oracle requirement) - the key small-model failure.
over-scaling	Unnecessary scale-ups (replicas above oracle requirement).
scale-up recall / precision	Decision quality: does the controller scale up when it should (recall) and only then (precision).
hold correctness	Does the controller leave well-enough alone when no change is needed.
MAE vs intended	Fidelity: distance from the ideal capacity-model decision.
latency / tokens	Tier-2 cost: wall-clock and token cost of the LLM decision.

6. LLMs, prompt, tools, and calls

Models (Ollama cloud, localhost, OpenAI-compatible, temp 0): glm-5.3-flash:cloud, glm-5.3:cloud, deepseek-v4-pro:cloud

Note: mistral-large-4 is excluded — ~3x slower and ~2x more tokens than the others while returning the same decisions.

System prompt (tool mode):

```
You are the Tier-2 compiler of a 6G network-slicing control plane. You receive per-slice telemetry
(eMBB, URLLC, mMTC): load (Mbps), current replicas, current delay/loss, KPI targets, and ML signals.
Use the provided tools to compute the correct replica count. Then act: call scale_replicas once per
slice (eMBB, URLLC, mMTC) with the replica count you decide. Do not output text or JSON - your
scale_replicas calls are the answer.
```

tool	does
estimate_capacity(slice, load)	load -> required replicas (capacity model)
scale_replicas(slice, n)	validate + clamp n to [1,5]
set_resources(slice, cpu, mem)	resource-limit manifest patch
add_nf_instance(slice, region)	register a new NF instance
apply_k8s_manifest(id, yaml)	validate a Deployment manifest
inspect_infrastructure(state)	utilization, capacity, headroom, required, budgets per slice

LLM calls (single-shot, 4 states per model):

model	call	state	eMBB	URLLC	mMTC	lat(ms)	tokens	reason
flash	1	10	1	2	3	2702.40	1178	mMTC violates delay (0.00426>0
flash	2	40	3	4	1	3525.00	1482	eMBB and URLLC both violate de
flash	3	70	3	4	1	1907.20	968	eMBB delay ~1.7x target & loss
flash	4	90	3	4	1	3108.10	1346	eMBB delay 1.67x target with 0
glm-5.3	1	10	1	2	5	4347.20	1299	mMTC severely violates KPIs (l
glm-5.3	2	40	3	4	1	6143.40	1694	eMBB and URLLC badly violate d
glm-5.3	3	70	4	5	1	4336.80	1191	eMBB and URLLC massively viola
glm-5.3	4	90	4	5	1	3402.90	1140	eMBB and URLLC both breach del
deepseek	1	10	1	1	3	13055.70	2779	mMTC KPI loss/delay violation,
deepseek	2	40	5	5	1	6373.70	1602	eMBB and URLLC severely violat
deepseek	3	70	5	5	1	3884.90	1140	High p_violation and severe de
deepseek	4	90	5	5	1	5389.60	1422	eMBB and URLLC are severely vi

7. Function-call traces (sample)

glm-5.3-flash:cloud (state i=0) — final actions {'eMBB': 2, 'URLLC': 1, 'mMTC': 1}

tool call	arguments	result
inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 2	ok
scale_replicas	URLLC -> 1	ok
scale_replicas	mMTC -> 1	ok

deepseek-v4-pro:cloud (state i=0) — final actions {'eMBB': 1, 'URLLC': 1, 'mMTC': 1}

tool call	arguments	result
inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 1	ok
scale_replicas	URLLC -> 1	ok
scale_replicas	mMTC -> 1	ok

glm-5.3:cloud (state i=0) — final actions {'eMBB': 5, 'URLLC': 5, 'mMTC': 5}

tool call	arguments	result

inspect_infrastructure	state (3 slices)	eMBB: util=0.368 req=1 URLLC: util=0.069 req=1 mMTC: util=0.154 req=1
scale_replicas	eMBB -> 5	ok
scale_replicas	URLLC -> 5	ok
scale_replicas	mMTC -> 5	ok
scale_replicas	eMBB -> 5	ok

8. Benchmark (per-slice closed loop)

Single run per method; LLM rows are one trajectory each — no repetitions, no error bars.

Scenario: **ramp**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	24.67	2.50	1.11	0.11	0.12	0.32	0.46	0.49
hpa	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
predict-then-threshold	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
capacity-rule	1.00	3.73	0.16	0.38	0.02	0.09	0.97	1.10
llm-flash	10.00	3.09	0.74	0.33	0.15	0.17	0.57	0.47
llm-glm5.3	8.00	3.50	0.57	0.57	0.17	0.18	0.64	0.87
llm-deepseek	13.33	2.83	0.86	0.19	0.12	0.18	0.59	0.21

Scenario: **steady**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	27.50	2.58	1.28	0.22	0.12	0.30	0.17	0.59
hpa	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
predict-then-threshold	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
capacity-rule	0.83	4.28	0.12	0.76	0.11	0.22	0.96	1.65
llm-flash	11.25	3.15	0.85	0.36	0.13	0.16	0.43	0.52
llm-glm5.3	9.17	3.47	0.72	0.55	0.20	0.19	0.37	0.84
llm-deepseek	16.67	2.72	1.11	0.19	0.11	0.20	0.42	0.13

Scenario: **spike**

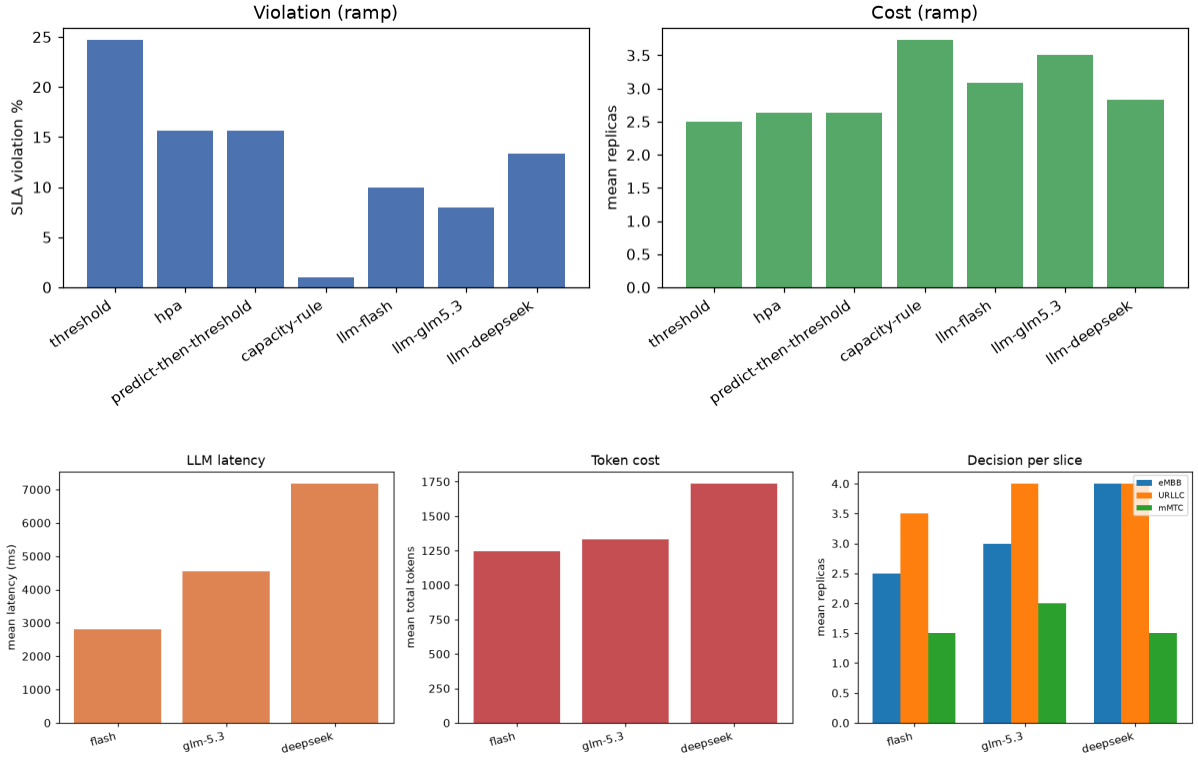
method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	20.00	2.15	1.11	0.19	0.11	0.36	0.52	0.44
hpa	14.58	2.24	0.95	0.12	0.15	0.33	0.62	0.00
predict-then-threshold	15.00	2.23	0.96	0.12	0.14	0.31	0.62	0.01
capacity-rule	1.67	4.06	0.22	1.21	0.09	0.33	0.95	1.82
llm-flash	8.33	2.68	0.70	0.31	0.17	0.27	0.69	0.44
llm-glm5.3	5.42	3.34	0.57	0.85	0.28	0.25	0.68	1.10
llm-deepseek	12.08	2.62	0.81	0.36	0.20	0.28	0.63	0.43

9. RQ5 — estimate_capacity tool

mode	viol%	repl	mae	under	over
no_tool	3.33	3.53	1.22	0.50	0.48
tool	10.00	2.57	0.22	1.12	0.13

Finding: estimate_capacity cuts MAE 1.22->0.22 and cost 3.53->2.57, but raises violation 3.3%->10.0% (exact, not safer). We therefore use inspect_infrastructure — the full per-slice state view — as the primary tool.

10. Comparison graphs


Violation (ramp)
SLA violation %
0
5
10
15
20
25
threshold
hpa
predict-then-threshold
capacity-rule
llm-flash
llm-glm5.3
llm-deepseek
Cost (ramp)
mean replicas
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
threshold
hpa
predict-then-threshold
capacity-rule
llm-flash
llm-glm5.3
llm-deepseek
LLM latency
mean latency (ms)
0
1000
2000
3000
4000
5000
6000
7000
flash
glm-5.3
deepseek
Token cost
mean total tokens
0
250
500
750
1000
1250
1500
1750
flash
glm-5.3
deepseek
Decision per slice
mean replicas
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
eMBB
URLLC
mMTC
flash
glm-5.3
deepseek